

발행 정책 서비스 + 조건 판단 Strategy

IssuancePolicyService · EventConditionService
DB 기반 정책 관리 · Strategy 패턴 · false positive

IssuancePolicyService

Strategy 패턴

PostgreSQL

COINCRAFT
ACADEMY

발행 전에 답해야 할 두 가지 질문

NFT 발행을 결정하는 두 개의 독립 관문

이벤트 · 사용자가 1만보를 걸었다 · 사용자가 건강검진을 받았다 · 사용자가 쿠폰을 사용했다

질문 1

이 이벤트에 대해 NFT를 발행할 수 있는가?

→ **IssuancePolicyService** 담당

- 정책 존재 여부
- 기간 유효성
- 수량 한도

질문 2

이 사용자가 조건을 충족하는가?

→ **EventConditionService** 담당

- 실제 이벤트 데이터 검증
- 사용자별 충족 여부

두 질문의 역할 — 왜 분리하는가

변경 이유가 다르면 분리한다 (SRP)

항목	IssuancePolicyService	EventConditionService
질문	발행 가능한가?	조건을 충족하는가?
데이터	DB (issuance_policy)	이벤트 payload
변경 주체	운영팀·기획자	개발자
변경 빈도	가끔 (정책 수정)	이벤트 추가 시
실패 시	NoPolicyError	false 반환

정책은 DB 관리자가 바꾸고, 조건은 개발자가 바꾼다 → 변경 이유가 다르면 클래스도 달라야 한다

발행 정책 서비스 — DB 기반 관리 (1/4)

안티패턴 — 정책을 코드에 하드코딩

안티패턴 코드

```
// ❌ 정책이 코드에 박혀 있다
if (eventType === 'WALKING_10000') {
  tokenId = 1001n;
  maxPerUser = 1;
} else if (eventType === 'HEALTH_CHECK') {
  tokenId = 1002n;
  maxPerUser = 1;
} else if (eventType === 'COUPON_SUMMER') {
  tokenId = 1003n;
  maxPerUser = 3;
}
```

문제점

- 1 정책 변경 = 코드 수정 + 배포
- 2 총 발행량·기간 조건 추가 시 분기 폭발
- 3 운영팀이 직접 수정 불가
- 4 테스트에서 정책을 독립 제어 불가

배포 없이 정책을 바꿀 수 없다 → issuance_policy 테이블로 이관

발행 정책 서비스 — DB 기반 관리 (2/4)

issuance_policy 테이블 설계

```
CREATE TABLE issuance_policy (  
  id          BIGSERIAL PRIMARY KEY,  
  token_id    BIGINT NOT NULL UNIQUE,  
  event_type  VARCHAR(64) NOT NULL UNIQUE,  
  name        VARCHAR(128) NOT NULL,  
  max_per_user INT NOT NULL DEFAULT 1,  
  total_supply INT, -- NULL = 무제한  
  start_date  TIMESTAMPTZ NOT NULL,  
  end_date    TIMESTAMPTZ NOT NULL,  
  metadata_uri VARCHAR(512) NOT NULL,  
  is_active   BOOLEAN NOT NULL DEFAULT true,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
CREATE INDEX idx_policy_event_type ON issuance_policy (event_type);
```

UNIQUE(event_type) — 이벤트 타입당 정책은 하나. token_id도 UNIQUE — 정책마다 고유 NFT 타입.

발행 정책 서비스 — DB 기반 관리 (3/4)

컬럼 설계 이유

컬럼	타입	설계 이유
token_id	BIGINT UNIQUE	ERC-1155 토큰 타입 식별자. 정책 1개 = NFT 타입 1개
event_type	VARCHAR UNIQUE	조건 판단 서비스의 라우팅 키
max_per_user	INT DEFAULT 1	사용자당 최대 보유 수. 1이면 중복 발행 방지
total_supply	INT NULLABLE	NULL = 무제한. 한정판 이벤트에만 설정
start/end_date	TIMESTAMP TZ	서버 시각 기준 유효 기간. 기간 외 요청 → NoPolicyError
is_active	BOOLEAN	긴급 비활성화. 배포 없이 정책 중단 가능

is_active + 기간 검증은 서비스 레이어에서 한다. DB에서 WHERE is_active = true로 필터링해도 되지만, 이유가 다른 실패를 구분하려면 코드에서 분기한다.

발행 정책 서비스 — DB 기반 관리 (4/4)

PglIssuancePolicyRepository — 조회 쿼리

findByEventType

```
async findByEventType(  
  eventType: string,  
) : Promise<IssuancePolicy | null> {  
  const res = await this.pool.query(  
    `SELECT id, token_id, event_type, name,  
      max_per_user, total_supply,  
      start_date, end_date,  
      metadata_uri, is_active  
    FROM issuance_policy  
    WHERE event_type = $1`,  
    [eventType],  
  );  
  if (!res.rowCount) return null;  
  return this._toModel(res.rows[0]);  
}
```

is_active + 기간 검증 (서비스 레이어)

```
async getPolicy(  
  eventType: string,  
) : Promise<IssuancePolicy> {  
  const p = await this.repo  
    .findByEventType(eventType);  
  
  if (!p || !p.isActive)  
    throw new NoPolicyError(eventType);  
  
  const now = new Date();  
  if (now < p.startDate || now > p.endDate)  
    throw new NoPolicyError(eventType);  
  
  return p;  
}
```

IssuancePolicyService (1/3)

interface IssuancePolicyRepository — DIP 경제

IssuancePolicyRepository

```
interface IssuancePolicyRepository {  
  findByEventType(  
    eventType: string,  
  ): Promise<IssuancePolicy | null>;  
  
  findById(  
    id: number,  
  ): Promise<IssuancePolicy | null>;  
}
```

DIP — 서비스는 인터페이스만 안다.

PgIssuancePolicyRepository (운영) /

InMemoryIssuancePolicyRepository (테스트) 중 어느 것이든 주입 가능

구현체 구조

PgIssuancePolicyRepository

implements IssuancePolicyRepository

→ pool.query() / _toModel()

InMemoryIssuancePolicyRepository

implements IssuancePolicyRepository

→ Map<string, IssuancePolicy> / 테스트용

IssuancePolicyService (2/3)

interface IssuancePolicy — 도메인 모델

```
interface IssuancePolicy {  
  id:          number;  
  tokenId:     bigint;  
  eventType:   string;  
  name:        string;  
  maxPerUser:  number;  
  totalSupply: number | null;  
  startDate:   Date;  
  endDate:     Date;  
  metadataUri: string;  
  isActive:    boolean;  
}
```

_toModel — DB row → 도메인

```
private _toModel(row: any): IssuancePolicy {  
  return {  
    id:          row.id,  
    tokenId:     BigInt(row.token_id),  
    eventType:   row.event_type,  
    name:        row.name,  
    maxPerUser:  row.max_per_user,  
    totalSupply: row.total_supply,  
    startDate:   new Date(row.start_date),  
    endDate:     new Date(row.end_date),  
    metadataUri: row.metadata_uri,  
    isActive:    row.is_active,  
  };  
}
```

tokenId: BigInt(row.token_id) — PostgreSQL BIGINT는 JS에서 string으로 온다. BigInt로 변환해야 온체인 ABI 인코딩에서 손실 없음.

IssuancePolicyService (3/3)

class NoPolicyError · class IssuancePolicyService

NoPolicyError

```
export class NoPolicyError extends Error {
  constructor(eventType: string) {
    super(`발행 정책 없음: ${eventType}`);
    this.name = 'NoPolicyError';
  }
}
```

IssuancePolicyService

```
export class IssuancePolicyService {
  constructor(
    private readonly repo: IssuancePolicyRepository,
  ) {}

  async getPolicy(
    eventType: string,
  ): Promise<IssuancePolicy> {
    const p = await this.repo
      .findByEventType(eventType);
    if (!p || !p.isActive)
      throw new NoPolicyError(eventType);
    const now = new Date();
    if (now < p.startDate || now > p.endDate)
      throw new NoPolicyError(eventType);
    return p;
  }
}
```

실패 경로 3가지

조건	에러
DB에 정책 없음	NoPolicyError
is_active = false	NoPolicyError
기간 벗어남	NoPolicyError

모두 NoPolicyError로 통일 — 컨트롤러에서 단일 catch로 처리

GetPolicy는 '발행 가능한가'만 답한다. 사용자가 조건을 충족하는지는 EventConditionService의 책임.

조건 판단 — 안티패턴

if-else 단일 함수로 모든 이벤트 처리

안티패턴 코드

```
async function checkCondition(  
  userId: string,  
  eventType: string,  
  payload: unknown,  
): Promise<boolean> {  
  if (eventType === 'WALKING_10000') {  
    const { stepCount } = payload as any;  
    return stepCount >= 10_000;  
  }  
  if (eventType === 'HEALTH_CHECK_ANNUAL') {  
    const { checkDate } = payload as any;  
    return isWithinCurrentYear(checkDate);  
  }  
  if (eventType.startsWith('COUPON_')) {  
    const { couponCode } = payload as any;  
    return externalApi.validateCoupon(  
      userId, couponCode);  
  }  
  return false; // 조용히 false - 위험  
}
```

문제점

- 1 이벤트 추가 = 이 파일 수정 (SRP 위반)
- 2 외부 의존성이 함수 안에 하드와이어드 → 테스트 불가
- 3 조용한 false — 지원하지 않는 이벤트가 그냥 통과
- 4 조건 로직이 커질수록 if 블록이 수백 줄로 팽창

OCP 위반 — 새 이벤트 추가 시 기존 함수를 수정한다

조건 판단 — Strategy 패턴

정책이 "무엇을" 결정한다면, 조건은 "할 수 있는가"를 결정

정책 (IssuancePolicyService)

- 이 이벤트의 tokenId는?
- 기간이 유효한가?
- maxPerUser는?

조건 (EventConditionChecker)

- 이 사용자가 실제로 1만보를 걸었는가?
- 검진을 받았는가?
- 쿠폰이 유효한가?

오케스트레이터 (IssuerService)

정책 통과? + 조건 통과?
→ 발행

Strategy 패턴 핵심 아이디어

각 이벤트의 조건 판단 로직을 별도 클래스(Checker)로 캡슐화한다. EventConditionService는 Checker 목록을 받아 eventType에 맞는 것을 찾아 위임한다.

새 이벤트 추가 = 새 Checker 클래스 작성 + 등록. 기존 코드 수정 없음 (OCP)

Strategy 패턴 — 공통 인터페이스 (1/5)

EventConditionChecker · EvaluationContext — 모든 Checker가 구현하는 계약

EvaluationContext

```
interface EvaluationContext {  
  userId:    string;  
  eventType: string;  
  payload:   unknown;  
}
```

EventConditionChecker

```
interface EventConditionChecker {  
  /** 이 Checker가 eventType을 처리할 수 있는가 */  
  supports(eventType: string): boolean;  
  /** 조건 충족 여부 반환 */  
  check(ctx: EvaluationContext): Promise<boolean>;  
}
```

supports()가 라우팅을 결정한다. EventConditionService는 supports()=true인 Checker에게 check()를 위임한다.

두 메서드의 역할 분리

메서드	역할
supports()	내가 이 이벤트를 처리할 수 있는가? (라우팅)
check()	실제 조건 평가 (비즈니스 로직)

payload: unknown — 각 Checker가 자신이 아는 타입으로 캐스팅. 외부로 타입 의존성 노출 없음.

Strategy 패턴 — 활동 기반 Checker (2/5)

WalkingConditionChecker · HealthCheckConditionChecker

WalkingConditionChecker

```
export class WalkingConditionChecker
  implements EventConditionChecker {

  supports(eventType: string): boolean {
    return eventType === 'WALKING_10000';
  }

  async check(
    ctx: EvaluationContext,
  ): Promise<boolean> {
    const { stepCount } =
      ctx.payload as { stepCount: number };
    return stepCount >= 10_000;
  }
}
```

HealthCheckConditionChecker

```
export class HealthCheckConditionChecker
  implements EventConditionChecker {

  supports(eventType: string): boolean {
    return eventType === 'HEALTH_CHECK_ANNUAL';
  }

  async check(
    ctx: EvaluationContext,
  ): Promise<boolean> {
    const { checkDate } =
      ctx.payload as { checkDate: string };
    const year = new Date(checkDate)
      .getFullYear();
    return year === new Date().getFullYear();
  }
}
```


Strategy 패턴 — 외부 자격 조회 Checker (3/5)

CouponConditionChecker — 외부 서비스 의존성을 생성자로 주입

```
export class CouponConditionChecker
  implements EventConditionChecker {

  constructor(
    private readonly couponAdapter: CouponAdapter,
  ) {}

  supports(eventType: string): boolean {
    return eventType.startsWith('COUPON_');
  }

  async check(
    ctx: EvaluationContext,
  ): Promise<boolean> {
    const { couponCode } =
      ctx.payload as { couponCode: string };
    return this.couponAdapter.isValid(
      ctx.userId,
      couponCode,
    );
  }
}
```

외부 의존성(CouponAdapter)은 생성자로 주입. 테스트에서 Mock으로 교체 가능.

CouponAdapter 인터페이스

```
interface CouponAdapter {
  isValid(
    userId: string,
    couponCode: string,
  ): Promise<boolean>;
}
```

Adapter = 외부 시스템 래퍼. REST API, 메시지큐, gRPC 구현체를 갈아 끼워도 Checker 코드는 변경 없음.

Strategy 패턴 — EventConditionService (4/5)

Checker 목록을 받아 eventType에 맞는 Checker에 위임

EventConditionService

```
export class EventConditionService {
  constructor(
    private readonly checkers: EventConditionChecker[],
  ) {}

  async evaluate(
    ctx: EvaluationContext,
  ): Promise<boolean> {
    const checker = this.checkers
      .find(c => c.supports(ctx.eventType));

    if (!checker)
      throw new UnsupportedEventTypeError(
        ctx.eventType,
      );

    return checker.check(ctx);
  }
}
```

DI 컨테이너 등록 예시

```
const conditionService = new EventConditionService([
  new WalkingConditionChecker(),
  new HealthCheckConditionChecker(),
  new CouponConditionChecker(couponAdapter),
]);
```

Checker 배열이 곧 지원 이벤트 목록. 순서는 supports() 매칭 우선순위.

UnsupportedEventTypeError → HTTP 400. 조용한 false보다 명시적 실패가 안전하다.

Strategy 패턴 — 새 이벤트 추가 (5/5)

기존 코드 수정 없이 Checker 하나만 추가

추가: 보험 가입 이벤트

```
// 새 파일: InsuranceConditionChecker.ts
export class InsuranceConditionChecker
  implements EventConditionChecker {

  supports(eventType: string): boolean {
    return eventType === 'INSURANCE_NEW';
  }

  async check(
    ctx: EvaluationContext,
  ): Promise<boolean> {
    const { contractId } =
      ctx.payload as { contractId: string };
    // 보험 계약 유효성 검증
    return contractId.startsWith('KL');
  }
}
```

변경 범위

- 1 InsuranceConditionChecker.ts 신규 작성
- 2 DI 등록 배열에 추가
- 3 기존 EventConditionService 수정 없음
- 4 기존 WalkingConditionChecker 수정 없음
- 5 기존 CouponConditionChecker 수정 없음

OCP (개방-폐쇄 원칙) — 확장에는 열려 있고, 수정에는 닫혀 있다

적용 전 — Monolithic 구조

조건 판단 로직이 한 함수에 집중

checkCondition.ts - 단일 파일에 모든 이벤트 로직 집중

```
async function checkCondition(userId, eventType, payload): Promise<boolean>
```

```
if eventType ===  
'WALKING_10000'
```

```
stepCount >= 10_000  
// 외부 의존성 없음 - 단위 테스트 가능
```

```
if eventType ===  
'HEALTH_CHECK_ANNUAL'
```

```
isWithinCurrentYear(checkDate)  
// 외부 의존성 없음
```

```
if eventType  
.startsWith('COUPON_')
```

```
externalApi.validateCoupon(userId, couponCode)  
⚠ 외부 의존성 하드와이어드 - 테스트에서 교체 불가
```

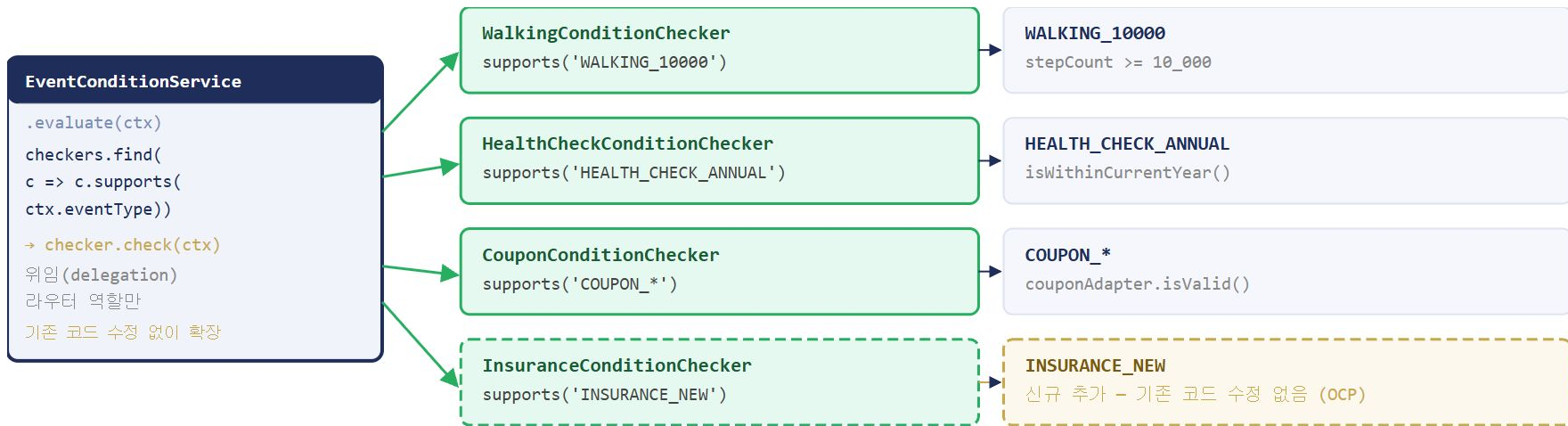
```
if eventType ===  
'INSURANCE_NEW'
```

```
신규 이벤트 추가 → 이 파일 수정 필수  
OCP 위반 · 이벤트 N개 = if 블록 N개 · 파일이 계속 팽창
```

단일 파일에 모든 이벤트 로직. 이벤트 10개 = if 블록 10개. 테스트에서 외부 의존성 격리 불가.

적용 후 — Strategy 패턴

각 Checker가 독립적으로 존재. EventConditionService는 라우터만



OCP 달성

새 이벤트: 파일 1개 추가 + DI 등록 1줄. 기존 코드 수정 없음.

테스트 격리

각 Checker를 독립 단위 테스트. Mock 없이도 순수 로직 검증 가능.

false positive vs false negative

조건 판단 오류의 두 방향 — 어느 쪽이 더 위험한가

구분	정의	예시	결과	위험도
false positive	조건 미충족자에게 NFT 발행	1만보 미달인데 발행	공정성 훼손·감사 위험·블록체인 기록 영구화	● 높음
false negative	조건 충족자에게 NFT 미발행	1만보 달성했는데 미발행	사용자 불만·재발행 비용	● 중간

false positive가 더 위험하다. 블록체인에 발행된 NFT는 취소가 없다 (burn은 가능하나 비용·UX 최악). 조건 판단은 보수적으로 — 애매하면 false.

그러므로 check() 구현 시: 예외 = false가 아니라 throw. `UnsupportedEventTypeError` = false 처리 금지.

두 서비스의 협력

IssuerService — 정책 + 조건 통과 시 발행 (S29 구현)

```
class IssuerService {
  async issue(
    userId: string, eventType: string, payload: unknown,
  ): Promise<void> {
    // 1. 정책 확인 (IssuancePolicyService)
    const policy = await this.policyService
      .getPolicy(eventType); // throws NoPolicyError

    // 2. 조건 확인 (EventConditionService)
    const ok = await this.conditionService
      .evaluate({ userId, eventType, payload });

    if (!ok) throw new ConditionNotMetError(userId, eventType);

    // 3. 중복 발행 방지 (S29)
    // 4. 온체인 Mint (S29)
  }
}
```

S28에서는 1-2단계를 완성한다. 3-4단계(중복 방지 + 온체인 Mint)는 S29에서.

두 서비스는 서로를 모른다. IssuerService만 두 서비스를 알고 있다 → 단방향 의존.

tokenId 출처 — IssuancePolicyService에서 온다 (1/2)

tokenId는 조건 판단이 아니라 정책 조회 결과

tokenId는 issuance_policy.token_id 컬럼 값. 이벤트 타입이 결정되면 정책이 토큰 타입을 지정한다.

이벤트 발생

eventType = 'WALKING_10000'

IssuancePolicyService.getPolicy()

```
SELECT token_id FROM issuance_policy  
WHERE event_type = $1
```

policy.tokenId = 1001n

ERC-1155 토큰 타입 식별자 확정

IssuerService

정책 + 조건 통과 확인 후 발행 진행

blockchain.mint(walletAddr, 1001n, 1n)

왜 Strategy가 tokenId를 모르는가

- 1 Checker의 책임: 조건 충족 여부 (true/false)
- 2 tokenId 결정: 정책의 책임 (DB)
- 3 두 책임을 섞으면 SRP 위반

Checker가 tokenId를 반환하면 — 새 이벤트 추가 시 tokenId도 Checker 코드에 하드코딩된다. DB 정책 변경이 코드 변경을 요구하게 됨.

tokenId — ERC-1155 토큰 타입 식별자 (2/2)

정책 1개 = NFT 타입 1개 = tokenId 1개

ERC-1155 발행 구조

event_type	token_id	설명
WALKING_10000	1001	만보 달성 NFT
HEALTH_CHECK_ANNUAL	1002	연간 건강검진 NFT
COUPON_SUMMER_2025	1003	여름 쿠폰 NFT
INSURANCE_NEW	1004	신규 가입 NFT

ERC-1155: 같은 tokenId를 여러 사용자가 보유 가능. 1인 1개 제한은 max_per_user = 1로 서비스 레이어에서 강제.

tokenId ≠ 순번

ERC-721처럼 순번이 아니다. ERC-1155의 tokenId는 '타입 식별자'. 같은 이벤트를 달성한 모든 사용자가 동일한 tokenId의 NFT를 보유한다.

mint 호출

```
// ERC-1155 safeTransferFrom / mint
await contract.mint(
  userWalletAddr,
  policy.tokenId, // 1001n
  1n,             // amount
  '0x',           // data
);
```

테스트 전략 (1/3)

조건 평가 로직은 완전한 테스트 커버리지가 필요

테스트 케이스 분류

Checker	케이스	기대값
WalkingConditionChecker	stepCount = 10000	true
WalkingConditionChecker	stepCount = 9999	false
WalkingConditionChecker	stepCount = 0	false
HealthCheckConditionChecker	올해 검진	true
HealthCheckConditionChecker	작년 검진	false
CouponConditionChecker	유효 쿠폰	true (Mock)
CouponConditionChecker	만료 쿠폰	false (Mock)

경계값 중심 테스트

false positive 방지를 위해 경계값(9999 vs 10000)을 반드시 테스트한다. 10000이 통과하고 9999가 실패해야 한다.

9999에서 true가 나오면 false positive → 실제 NFT 오발행

stepCount = 10000 정확히 통과 — 이상 조건 ($\geq$). 미만이면 false.

테스트 전략 (2/3)

테스트 더블 구분 — 무엇을 Mock하고 무엇을 실제로 쓰는가

대상	더블 종류	이유
IssuancePolicyRepository	Stub	findByEventType()가 고정 정책 반환. DB 연결 불필요
CouponAdapter	Mock	isValid() 호출 여부·인자 검증 필요
WalkingConditionChecker	실제 객체	외부 의존성 없음. 단위 테스트에서 실 코드 사용
EventConditionService	실제 객체	Checker 배열 주입. 통합 시나리오 검증
pool (PostgreSQL)	실제 DB TestContainer	Repository 통합 테스트

외부 의존성이 없는 Checker는 Mock 없이 테스트. 외부 호출이 있는 Checker(CouponConditionChecker)만 Mock.

테스트 전략 (3/3)

AAA 패턴 — Arrange · Act · Assert

단위 테스트 예시 (경계값)

```
describe('WalkingConditionChecker', () => {
  const checker = new WalkingConditionChecker();

  it('stepCount 10000 → true', async () => {
    // Arrange
    const ctx: EvaluationContext = {
      userId: 'U001', eventType: 'WALKING_10000',
      payload: { stepCount: 10_000 },
    };
    // Act
    const result = await checker.check(ctx);
    // Assert
    expect(result).toBe(true);
  });

  it('stepCount 9999 → false', async () => {
    const ctx = { ...baseCtx,
      payload: { stepCount: 9_999 } };
    expect(await checker.check(ctx)).toBe(false);
  });
});
```

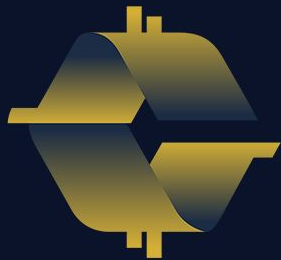
Mock 주입 테스트 (CouponChecker)

```
it('유효 쿠폰 → true', async () => {
  // Arrange
  const mockAdapter: CouponAdapter = {
    isValid: jest.fn().mockResolvedValue(true),
  };
  const checker =
    new CouponConditionChecker(mockAdapter);
  const ctx: EvaluationContext = {
    userId: 'U001',
    eventType: 'COUPON_SUMMER',
    payload: { couponCode: 'SUMMER2025' },
  };
  // Act
  const result = await checker.check(ctx);
  // Assert
  expect(result).toBe(true);
  expect(mockAdapter.isValid)
    .toHaveBeenCalledWith('U001', 'SUMMER2025');
});
```

실습

S28 실습 — 7개 섹션

- [1] IssuancePolicyService — DB 정책 조회 & NoPolicyError
- [2] EventConditionService — 미등록 이벤트 → eligible=false
- [3] ActivityConditionStrategy — 걸음수 검증 (경계값 포함)
- [4] ActivityConditionStrategy — tokenId 비트 인코딩
- [5] CouponConditionStrategy — eligibility 외부 서비스 조회
- [6] PremiumConditionStrategy — threshold 비교
- [7] OCP 검증 — 새 전략 등록 시 서비스 코드 수정 없음



COINCRAFT
ACADEMY

[1] IssuancePolicyService — DB 정책 조회

npm run exercise:s28:1 · issuance_policies UPSERT → SELECT → NoPolicyError

```
$ npm run exercise:s28:1
```

실행 코드

```
const svc = new IssuancePolicyService(pool);
const walkTokenId =
  (BigInt(0x01) << BigInt(64)) | BigInt(1);
await svc.registerPolicy({
  eventType: 'WALK_GOAL_MET',
  tokenId: walkTokenId, amount: 1n
});
const policy = await svc.getPolicy('WALK_GOAL_MET');
// 미등록 정책 → NoPolicyError
try {
  await svc.getPolicy('UNKNOWN_EVENT');
} catch (err) {
  console.log(err.constructor.name);
  // NoPolicyError
}
```

예상 출력

```
[DB] UPSERT issuance_policies
      event_type='WALK_GOAL_MET'  amount=1
[DB] SELECT token_id, amount
      WHERE event_type='WALK_GOAL_MET'
      → rowCount=1
      → tokenId=18446744073709551617  amount=1
미등록 이벤트 에러: NoPolicyError
```

핵심 포인트

issuance_policies 테이블:

event_type UNIQUE 제약 → UPSERT로 중복 방지

IssuerService는 직접 DB 접근 금지 — 이 서비스를 통해 간접 접근

NoPolicyError:

rowCount=0 → 즉시 throw

tokenId 조회조차 하지 않음 → 정책 없는 이벤트는 발행 불가

DB 사전 준비:

```
npm run exercise:s27:db:up
```

S27과 동일 DB 사용 — init-db.sql에 issuance_policies 테이블 포함

[2] EventConditionService — 미등록 이벤트 → eligible=false

npm run exercise:s28:2 · 전략 미등록 eventType → false (throw 아님)

```
$ npm run exercise:s28:2
```

실행 코드

```
const service = new EventConditionService();
// 전략 등록 없이 바로 evaluate
const result = await service.evaluate(
  makeEvent({ eventType: 'TOTALLY_UNKNOWN' })
);
console.log('eligible:', result.eligible);
console.log('reason:', result.reason);
```

예상 출력

```
[CONDITION] eventType='TOTALLY_UNKNOWN'
→ eligible=false (미등록 전략)
eligible: false ← false (예외 아님)
reason: unsupported event type: TOTALLY_UNKNOWN
```

핵심 포인트

false positive > false negative:

온체인 발행은 취소 불가 → 오발행이 미발행보다 훨씬 위험
미등록 이벤트 → throw 아닌 eligible=false 반환

전략 미등록 처리:

```
strategies.get(eventType) → undefined
→ { eligible: false, reason: 'unsupported...' }
```

등록된 전략이 있어야 평가 가능:

다음 섹션에서 registerStrategy()로
전략 등록 후 evaluate() 정상 동작 확인

[3] ActivityConditionStrategy — WALK_GOAL_MET 걸음수 검증

npm run exercise:s28:3 · steps >= 10,000 경계값 테스트

```
$ npm run exercise:s28:3
```

실행 코드

```
const service = new EventConditionService();
service.registerStrategy(
  new ActivityConditionStrategy());

const pass = await service.evaluate(
  makeEvent({ eventType: 'WALK_GOAL_MET',
    data: { steps: 15_000 } })
);
const fail = await service.evaluate(
  makeEvent({ eventType: 'WALK_GOAL_MET',
    data: { steps: 5_000 } })
);
const edge = await service.evaluate(
  makeEvent({ eventType: 'WALK_GOAL_MET',
    data: { steps: 10_000 } })
);
```

예상 출력

```
15000보: true ← true
5000보: false
  reason: steps 5000 < goal 10000
10000보 (경계값): true ← true (≥ 10000)
```

핵심 포인트

걸음수 목표 = 10,000보:

```
steps >= GOAL_STEPS (10_000)
```

경계값 10,000보 → true

(초과 아님, 이상 — ≥ not >)

registerStrategy() 등록:

```
supportedEventTypes = ['WALK_GOAL_MET', 'HEALTH_CHECK_DONE']
```

두 이벤트 모두 동일 전략 인스턴스가 처리

경계값 테스트 필수:

9,999보 → false (목표 미달)

10,000보 → true (달성)

10,001보 → true (초과 달성)

[4] ActivityConditionStrategy — tokenId 비트 인코딩

npm run exercise:s28:4 · (productCode << 64) | eventCode

```
$ npm run exercise:s28:4
```

실행 코드

```
const walkResult = await service.evaluate(
  makeEvent({
    eventType: 'WALK_GOAL_MET',
    data: { steps: 12_000 },
    eventCode: 5
  })
);
const expected =
  (BigInt(0x01) << BigInt(64)) | BigInt(5);
console.log(
  'WALK tokenId:', walkResult.tokenId
);
console.log(
  '일치:', walkResult.tokenId === expected
);
```

예상 출력

```
WALK   tokenId: 18446744073709551621
가넷값:      18446744073709551621
일치: true
구조: (productCode << 64) | eventCode
WALK   productCode = 0x01 = 1
HEALTH productCode = 0x02 = 2
```

핵심 포인트

tokenId 비트 구조:

상위 64비트: productCode (상품 타입)

하위 64비트: eventCode (이벤트 코드)

```
WALK = 0x01 << 64 HEALTH = 0x02 << 64
```

ERC-1155 토큰 타입 식별:

단일 컨트랙트에서 여러 토큰 타입 관리
tokenId 구조 = 자기 설명적 메타데이터

BigInt 산술 필수:

JavaScript `number` 는 53비트 정수만 정확
128비트 tokenId → 반드시 `BigInt` 사용

[5] CouponConditionStrategy — eligibility 외부 서비스 조회

npm run exercise:s28:5 · DIP — EligibilityChecker 인터페이스 의존

```
$ npm run exercise:s28:5
```

실행 코드

```
const stubChecker: EligibilityChecker = {
  async isEligible(userId) {
    console.log(`[ELIGIBILITY] isEligible`);
    return userId === 'K-eligible';
  },
};
service.registerStrategy(
  new CouponConditionStrategy(stubChecker)
);
const pass = await service.evaluate(
  makeEvent({ eventType: 'COUPON_CLAIM',
    userId: 'K-eligible', eventCode: 10 })
);
const fail = await service.evaluate(
  makeEvent({ eventType: 'COUPON_CLAIM',
    userId: 'K-no-privilege', eventCode: 10 })
);
```

예상 출력

```
[ELIGIBILITY] isEligible(userId='K-eligible')
K-eligible    eligible: true ← true
[ELIGIBILITY] isEligible(userId='K-no-privilege')
K-no-privilege eligible: false
              reason: user not in eligibility list
```

핵심 포인트

DIP (의존성 역전):

EligibilityChecker 인터페이스에만 의존

실습: stub / 운영: 실제 API 클라이언트

전략 코드 변경 없이 구현체 교체 가능

외부 서비스 결과에 완전 위임:

isEligible() 응답이 유일한 기준

전략 내부에 별도 조건 로직 없음

테스트 용이성:

stub/mock으로 외부 서비스 없이

단위 테스트 가능

인터페이스가 테스트 경계를 만든다

[6] PremiumConditionStrategy — threshold 비교 (경계값 포함)

npm run exercise:s28:6 · amount >= threshold · BigInt 비교 · 누락 방어

```
$ npm run exercise:s28:6
```

실행 코드

```
service.registerStrategy(  
  new PremiumConditionStrategy(BigInt(100_000))  
);  
// 20만원 납부  
const pass = await service.evaluate(  
  makeEvent({ eventType: 'PREMIUM_PAID',  
    data: { amount: 200_000 } })  
);  
// 5만원 납부  
const fail = await service.evaluate(  
  makeEvent({ eventType: 'PREMIUM_PAID',  
    data: { amount: 50_000 } })  
);  
// amount 누락 → 0으로 처리  
const none = await service.evaluate(  
  makeEvent({ eventType: 'PREMIUM_PAID',  
    data: {} })  
);
```

예상 출력

```
20만원: true ← true  
5만원: false  
  reason: amount 50000 < threshold 100000  
10만원 (경계값): true ← true (≥ threshold)  
amount 누락: false ← false (0으로 처리)
```

핵심 포인트

threshold 생성자 주입:

이벤트별 다른 threshold 설정 가능

```
new PremiumConditionStrategy(BigInt(500_000))
```

전략 코드 변경 없이 기준금액 조정

BigInt 비교:

amount >= threshold — 정수 정확도 보장

금액 계산에 number 금지 (부동소수점 오차)

amount 누락 방어:

```
Number(event.data['amount']) ?? 0)
```

payload 없으면 0 → false (안전 방향)

[7] OCP 검증 — 새 전략 등록 시 서비스 코드 수정 없음

npm run exercise:s28:7 · registerStrategy() 1회 = 새 이벤트 지원 추가

```
$ npm run exercise:s28:7
```

실행 코드

```
const service = new EventConditionService();
service.registerStrategy(
  new ActivityConditionStrategy();

// 새 이벤트 - EventConditionService 코드 수정 없이
const loyaltyStrategy: IConditionStrategy = {
  supportedEventTypes: ['LOYALTY_REWARD'],
  async evaluate() {
    return {
      eligible: true,
      tokenId: BigInt(0xFF) << BigInt(64),
      amount: 1n
    };
  },
};
service.registerStrategy(loyaltyStrategy);
const result = await service.evaluate(
  makeEvent({ eventType: 'LOYALTY_REWARD' })
);
```

핵심 포인트

OCP (개방-폐쇄 원칙):

확장에 열려있고, 수정에 닫혀있다

새 이벤트 추가 = 새 클래스 + registerStrategy() 1줄

EventConditionService 코드 건드리지 않음

Map 덮어쓰기:

동일 eventType 재등록 → 나중 전략 우선

```
strategies.set(eventType, strategy)
```

전략 교체 시나리오:

기존 WALK 조건 변경 → 새 클래스 작성 후 재등록

기존 전략 코드, 서비스 코드 수정 없음

S28 · 발행 정책 서비스 + 조건 전략 패턴

Thank You

IssuancePolicyService · EventConditionService · Strategy 패턴
OCP · DIP · NoPolicyError · tokenId 비트 인코딩

M5 · S28

실습 완료

kyobo-digital-asset-platform

COINCRAFT
ACADEMY